

**Protocolo distribuido de control y reconfiguración topológica con continuidad lógica para
redes multisalto sobre Wi-Fi convencional**

Juan Carlos Clavijo Triviño

11162213503

Brandon Stiven Ganzo Murcia

11162217498

Universidad Antonio Nariño

Ingeniería de Sistemas y Computación

Facultad de Ingeniería de Sistemas

Bogotá D.C., Colombia

2026

**Protocolo distribuido de control y reconfiguración topológica con continuidad lógica para
redes multisalto sobre Wi-Fi convencional**

Juan Carlos Clavijo Triviño
Brandon Stiven Ganzo Murcia

Proyecto de grado presentado como requisito parcial para optar al título de:

Ingeniero de Sistemas y Computación

Director(a):
Elio Higinio Cables Pérez, Ph.D.

Codirector(a):

—

Línea de Investigación:
Una plataforma escalable de comunicación para escenarios post-desastre utilizando tecnologías
mesh y micro-computadores para redes ad hoc con cobertura ampliada

Universidad Antonio Nariño
Ingeniería de Sistemas y Computación
Facultad de Ingeniería de Sistemas
Bogotá D.C., Colombia

2026

Nota de aceptación

El trabajo de grado titulado *Protocolo distribuido de control y reconfiguración topológica con continuidad lógica para redes multisalto sobre Wi-Fi convencional* cumple con los requisitos para optar al título de *Ingeniero de Sistemas y Computación*.

Firma del Tutor

Firma Jurado

Firma Jurado

Bogotá D.C., Colombia, día mes 2026.

Dedicatoria

Agradecimientos

Índice

Lista de símbolos y abreviaturas

Símbolos con letras latinas

Abreviaturas

PCT	Protocolo de Control Topológico
GO	Group Owner (Wi-Fi Direct)
STA	Estación (modo infraestructura / legacy)
DNS-SD	DNS-Based Service Discovery
MANET	Mobile Ad Hoc Network
UUID	Identificador único universal
API	Application Programming Interface

Resumen

El presente documento describe el diseño y la validación empírica de un protocolo distribuido de control topológico para redes multisalto, enfocado en dispositivos móviles Android que utilizan interfaces Wi-Fi convencionales. El protocolo permite a los terminales descubrirse mutuamente, formar enlaces y organizar de forma autónoma una estructura de enrutamiento en entornos carentes de infraestructura centralizada (tales como routers o puntos de acceso físicos). Para asegurar la continuidad lógica de la comunicación, se definen mecanismos de reconfiguración que detectan la desconexión abrupta de nodos intermedios —un evento característico de escenarios post-desastre— y enrutan nuevamente el flujo de información, limitando los ciclos topológicos a través de tablas de estado locales guiadas por métricas de salto. La validación se llevará a cabo a nivel de capa de aplicación, probando su eficacia de resiliencia inter-nodo sin requerir modificaciones profundas a nivel de sistema operativo (root). Palabras clave: Protocolo distribuido, Wi-Fi convencional, Redes multisalto, Topología dinámica, Android, Comunicación post-desastre.

Abstract

This document details the design and empirical validation of a distributed topological control protocol for multi-hop networks, aimed at Android mobile devices using conventional Wi-Fi interfaces. The protocol enables terminals to mutually discover each other, establish links, and autonomously organize a routing structure in environments completely lacking centralized infrastructure (such as standard physical routers or access points). To ensure the logical continuity of communication, reconfiguration mechanisms are defined to detect the abrupt disconnection of intermediate nodes—an event characteristic of post-disaster scenarios—and dynamically reroute the information flow, preventing topological cycles through local state tables guided by hop metrics. The validation will be conducted at the application layer, testing its inter-node resilience without requiring extensive operating system modifications (root access). Keywords: Distributed protocol, Conventional Wi-Fi, Multi-hop networks, Dynamic topology, Android, Post-disaster communication.

Introducción

Las redes de telecomunicaciones modernas descansan sobre arquitecturas estrictamente centralizadas, una dependencia estructural que garantiza altas velocidades y estabilidad cuando las operadoras funcionan en condiciones nominales, pero que se convierte en una falla sistémica ante contextos de disrupción, tales como desastres naturales o apagones sostenidos. En estos escenarios, el colapso de las antenas, los puntos de acceso o el tendido de fibra óptica deja a múltiples comunidades aisladas, paradójicamente, a pesar de que los dispositivos inteligentes en sus manos poseen potentes interfaces radiales capaces de emitir y recibir información a nivel local. Este proyecto aborda la formulación algorítmica y estructural de una solución orientada a la creación de redes de emergencia (redes ad hoc multisalto), eliminando por definición la necesidad de coordinadores centrales fijos. El enfoque tecnológico aprovecha el espectro Wi-Fi nativo e inalterado en dispositivos con el sistema operativo Android, sorteando limitantes de acceso a las capas MAC que históricamente han marginado a las implementaciones de redes en malla (Mesh) hacia infraestructuras costosas, hardwares estandarizados por consorcios o móviles modificados en su núcleo (rooteados). El desarrollo se fundamenta en un enfoque metodológico iterativo que permite la validación progresiva de los algoritmos de enrutamiento y reconfiguración, buscando consolidar un protocolo que garantice la conectividad lógica en redes móviles descentralizadas. El presente documento se organiza en cinco capítulos: el Capítulo 1 define el planteamiento del problema, los objetivos y el alcance del proyecto; el Capítulo 2 establece el marco de referencia teórico, legal y el estado del arte; el Capítulo 3 detalla el componente metodológico basado en incrementos funcionales; finalmente, los Capítulos 4 y 5 presentan el cronograma de ejecución y el presupuesto requerido para la materialización de la investigación.

Antecedentes

Descripción del problema

En la actualidad, las redes de telecomunicaciones dependen principalmente de infraestructura centralizada, como torres celulares, routers o puntos de acceso que coordinan la comunicación entre los dispositivos. Este modelo permite gestionar la conectividad de forma eficiente en condiciones normales, sin embargo, produce una dependencia directa: cuando dicha infraestructura falla, se congestiona o queda fuera de servicio, la comunicación entre los usuarios puede verse interrumpida de manera parcial o total (Tanenbaum & Wetherall, 2011). Esta situación se vuelve especialmente relevante en escenarios como desastres naturales, fallas generalizadas de red o contextos donde la infraestructura de comunicaciones no está disponible. En estos casos, aunque los dispositivos móviles continúan funcionando y mantienen capacidades de conectividad inalámbrica, los usuarios pueden quedar aislados debido a la ausencia de sistemas que permitan coordinar la comunicación de forma directa entre los dispositivos cercanos (Akyildiz et al., 2005). A pesar de que tecnologías como Wi-Fi están ampliamente disponibles en teléfonos móviles y otros dispositivos electrónicos de uso cotidiano, su funcionamiento convencional está orientado principalmente a un modelo basado en infraestructura, donde un punto de acceso central gestiona el tráfico y la organización de la red (IEEE, 2020). Como resultado, el alcance efectivo de comunicación queda limitado al rango individual del punto central del Wi-Fi, que normalmente se encuentra aproximadamente entre 10 y 100 metros, dependiendo de las condiciones del entorno, la potencia de transmisión y las características del dispositivo (Gupta & Kumar, 2000). Esto conlleva a que dispositivos físicamente cercanos pierdan su capacidad de interconexión ante cualquier fallo, saturación o limitación en la cobertura del nodo central, quedando aislados a pesar de encontrarse dentro del rango radial de otros dispositivos. Las redes multisalto presentan una alternativa que permite que la comunicación se mantenga activa incluso si uno o varios nodos fallan, aumentando la resiliencia del sistema. Además, este enfoque resulta especialmente relevante en escenarios críticos, como desastres naturales, eventos masivos o zonas rurales sin infraestructura estable, donde la comunicación inmediata

y confiable es esencial (Akyildiz et al., 2005). Sin embargo, aunque existen protocolos multisalto, la mayoría requiere hardware especializado y configuraciones manuales de carácter complejo, especialmente a la hora de añadir nuevos nodos a la red, lo que limita su aplicabilidad en dispositivos de uso cotidiano (IEEE, 2020). En consecuencia, surge la necesidad de desarrollar un protocolo distribuido capaz de organizar dispositivos Wi-Fi convencionales en redes multisalto adaptativas, asegurando que la comunicación se mantenga aún en entornos dinámicos, extendiendo el alcance de comunicación y eliminando la dependencia de infraestructura centralizada.

Formulación del problema

Las redes inalámbricas convencionales dependen de infraestructura centralizada que, cuando falla o no está disponible, impide la comunicación entre dispositivos cercanos. Aunque los estándares IEEE 802.11 contemplan modos ad hoc y mesh (IEEE, 2020), las soluciones existentes requieren hardware especializado o modificaciones en las capas inferiores de la pila de protocolos, lo que las hace inviables en dispositivos móviles de uso cotidiano. Esto evidencia la necesidad de mecanismos de control distribuido que permitan a los dispositivos organizarse autónomamente, establecer relaciones topológicas coherentes y adaptarse a cambios en la conectividad operando exclusivamente sobre las interfaces estándar disponibles. A partir de esta necesidad se plantea la siguiente pregunta de investigación: ¿Cómo diseñar un protocolo distribuido que permita formar y mantener una topología multisalto mediante interfaces Wi-Fi estándar, asegurando estabilidad, prevención de ciclos y reconfiguración ante cambios dinámicos?

Justificación

El estudio de mecanismos alternativos de comunicación resulta relevante en contextos donde la infraestructura tradicional de telecomunicaciones no se encuentra disponible o presenta fallas.

En situaciones como desastres naturales, interrupciones del servicio o saturación de las redes, los sistemas centralizados pueden dejar de operar temporalmente, lo que provoca que los usuarios pierdan la posibilidad de comunicarse entre si (Akyildiz et al., 2005). Esta situación se intensifica en los escenarios de desastres naturales, donde la comunicación pasa a ser un elemento vital en la búsqueda de sobrevivientes y en la coordinación de las labores de rescate. Ante este tipo de escenarios, investigar formas de comunicación que no dependan de infraestructura fija puede aportar alternativas para mantener el intercambio de información entre dispositivos cercanos. Desde una perspectiva social, la posibilidad de que los dispositivos móviles puedan conectarse directamente entre si y extender la comunicación mediante saltos entre nodos puede resultar útil en entornos donde la conectividad es limitada o inexistente, como en zonas rurales, donde la infraestructura de telecomunicaciones puede ser escasa o costosa, de igual manera que en eventos masivos, donde un solo nodo central puede verse saturado por la cantidad de usuarios simultáneos (Appiahene et al., 2019). El uso de tecnologías que ya están integradas en la mayoría de los dispositivos, como el Wi-Fi, permite explorar soluciones que no requieren hardware adicional ni modificaciones complejas en la infraestructura existente, ni desde el lado de los dispositivos de los usuarios finales, como también de la infraestructura de telecomunicaciones móvil o fija. En el ámbito de la ingeniería de sistemas, el análisis de protocolos distribuidos aplicados a redes inalámbricas representa un campo de investigación importante, particularmente en temas relacionados con la autoorganización de nodos, el control de topologías dinámicas y la continuidad de la comunicación en redes con conectividad variable (Gupta & Kumar, 2000; Karl & Willig, 2005). Estudiar cómo estos mecanismos pueden implementarse sobre interfaces Wi-Fi estándar permite comprender mejor las posibilidades y limitaciones de este tipo de redes, especialmente ante el uso masivo moderno de la conectividad inalámbrica, misma que induce entropía en la topología de la red. En conjunto, el desarrollo de este trabajo se enmarca dentro del proyecto de investigación denominado (Una

plataforma escalable de comunicación para escenarios post-desastre utilizando tecnologías mesh y microcomputadores para redes ad hoc con cobertura ampliada). La investigación aporta al análisis de mecanismos de comunicación descentralizados que puedan operar utilizando tecnologías ampliamente disponibles, como el Wi-Fi en dispositivos móviles. Asimismo, permite generar una base conceptual y experimental que pueda servir como referencia para futuros desarrollos orientados a mejorar la resiliencia de las comunicaciones en escenarios de desastres naturales, donde la infraestructura de telecomunicaciones convencional no se encuentra disponible.

Objetivos

Objetivo general

Desarrollar un protocolo distribuido de control y reconfiguración de redes multisalto mediante interfaces Wi-Fi estándar, que permita a los nodos mantener comunicación estable, prevenir ciclos y adaptarse dinámicamente a cambios en la topología, mediante un diseño experimental en dispositivos Android.

Objetivos específicos

Diseñar el protocolo distribuido de control y reconfiguración topológica, definiendo mecanismos de descubrimiento de nodos, establecimiento de enlaces y prevención de ciclos. Definir la estructura de comunicación lógica entre nodos, incluyendo la forma en que los dispositivos intercambian información y mantienen la continuidad de la topología de manera autónoma. Implementar una propuesta conceptual del protocolo en dispositivos Android, mediante una aplicación demostrativa que permita la creación y reorganización dinámica de la red. Evaluar el comportamiento del protocolo mediante pruebas de laboratorio controladas, midiendo métricas de conectividad lógica, tiempo de reconfiguración ante fallos de nodo y continuidad del flujo de información en escenarios de topología variable.

Alcances

Los alcances del proyecto definen los límites lógicos y compromisos técnicos para el desarrollo del protocolo distribuido, estableciendo de manera clara los componentes que serán materializados en el sistema experimental y los resultados esperados al finalizar la investigación.

Diseño formal del protocolo: Un documento técnico detallando la arquitectura lógica del protocolo, que incluye la especificación de los mecanismos de descubrimiento de nodos, diagramas de estado para el establecimiento de enlaces y tablas de decisión para la prevención de ciclos. Aplicación Android demostrativa: Un paquete de instalación (APK) funcional para dispositivos móviles que integra el motor del protocolo mediante interfaces Wi-Fi estándar, habilitando la for-

mación de topologías dinámicas y un sistema de mensajería en tiempo real. Validación en laboratorio: Un informe de resultados empiricos que consolida la evaluación de la conectividad inicial, el tiempo de reconexión topológica ante fallos y las métricas de continuidad del flujo de información en escenarios controlados.

No entregables

Modificaciones a la capa física o a la capa MAC del estándar IEEE 802.11. Mecanismos de calidad de servicio (QoS) o cifrado extremo a extremo. Despliegue en producción o compatibilidad con sistemas operativos distintos de Android.

Limitaciones

Tiempo limitado a 12 semanas según calendario académico semestral, restringiendo iteraciones de prueba.

El desarrollo está restringido a dispositivos con ****Android 12 (API nivel 31)****, los cuales deben cumplir con los requisitos físicos mínimos del Compatibility Definition Document (CDD) de Google (Google, 2021): al menos 2 GB de memoria RAM (evitando la versión Go Edition para asegurar estabilidad), una resolución de pantalla mínima de 426 dp x 320 dp y soporte mandatorio para Wi-Fi Direct (P2P), esencial para la formación de la red multisalto sin infraestructura. Acceso a funciones (SoftAP, Wi-Fi Direct) sujeto a restricciones de las APIs de Android por fabricante, dificultando la portabilidad. Al operar en capa de aplicación, el protocolo hereda la latencia y restricciones energéticas del SO, limitando los tiempos de respuesta.

Marco teórico

Fundamentos conceptuales y tecnológicos

El desarrollo del presente proyecto requiere una comprensión sólida de los fundamentos conceptuales y tecnológicos que sustentan el diseño de un protocolo distribuido para redes multi-salto sobre Wi-Fi convencional. Este apartado tiene como finalidad proporcionar una visión integral de los conceptos relacionados con las redes inalámbricas, los protocolos de comunicación distribuida, las técnicas de enrutamiento y las herramientas tecnológicas empleadas en el desarrollo. A continuación, se presentan los elementos teóricos más relevantes, organizados en definiciones no técnicas y técnicas, con el propósito de contextualizar tanto la problemática abordada como las decisiones de diseño adoptadas en el protocolo propuesto.

Definiciones No Técnicas Se define como la relación entre fenómenos naturales peligrosos (terremotos, inundaciones, huracanes) y condiciones de vulnerabilidad física y socioeconómica de una comunidad (Maskrey, 1993, pp. 6–9). Para el proyecto, este concepto delimita el escenario de aplicación: entornos donde la infraestructura de comunicación convencional puede estar comprometida o destruida, y donde el protocolo propuesto debe operar de manera autónoma.

Resiliencia en Comunicaciones La resiliencia se entiende como la capacidad de un sistema para mantener o restaurar sus funciones esenciales frente a perturbaciones (Sterbenz et al., 2010). En este proyecto se materializa en la capacidad del protocolo para reorganizar la topología ante la pérdida de nodos, garantizando la continuidad de la comunicación sin intervención del usuario.

Comunicación Descentralizada Se refiere a los mecanismos de intercambio de información en los que no existe un nodo central que coordine el tráfico; cada dispositivo toma decisiones autónomamente con información local (Conti & Giordano, 2014). Este principio es el fundamento de diseño del protocolo propuesto, que permite a los nodos autoorganizarse sin depender de ningún coordinador externo.

Definiciones Técnicas Red de Área Local Inalámbrica (WLAN) y Estándar IEEE 802.11 Una WLAN emplea señales de radiofrecuencia para conectar dispositivos sin cableado; el estándar

IEEE 802.11 (Wi-Fi) define los protocolos de las capas física y MAC (IEEE, 2020). En su modo convencional opera con un punto de acceso centralizado cuyo rango oscila entre 10 y 100 metros (Gupta & Kumar, 2000), limitación que el protocolo propuesto busca superar operando en la capa de aplicación.

Red Ad Hoc y Redes Mesh Una red ad hoc es una red inalámbrica temporal y dinámica sin infraestructura preexistente, donde cada nodo actúa como cliente y como enrutador (Perkins, 2001). Las redes mesh evolucionan este concepto formando una malla con múltiples caminos alternativos que incrementa la robustez y la cobertura (Akyildiz et al., 2005), aunque su implementación comercial suele requerir hardware especializado que limita su uso en dispositivos móviles cotidianos (IEEE, 2020).

Una red multisalto permite la comunicación entre dispositivos fuera del rango directo mediante el reenvío de paquetes a través de nodos intermedios (Gupta & Kumar, 2000). Este mecanismo es el núcleo del protocolo propuesto: permite que dispositivos Android con Wi-Fi convencional se comuniquen sin conexión directa, y la cantidad de saltos refleja la distancia topológica con implicaciones directas en latencia y confiabilidad.

Un protocolo distribuido es un conjunto de reglas que permite a múltiples nodos coordinarse y tomar decisiones autónomamente sin controlador centralizado (Tanenbaum & Van Steen, 2007). En el proyecto, este componente habilita a los dispositivos Android para descubrir vecinos, establecer enlaces lógicos, prevenir ciclos y reorganizarse dinámicamente, convergiendo a un estado estable sin coordinación global.

Topología de Red y Gestión Topológica La topología describe cómo están interconectados los nodos; en redes dinámicas puede variar por movilidad, pérdida de enlaces o incorporación de nuevos dispositivos (Karl & Willig, 2005). La gestión topológica engloba los mecanismos para mantener una representación coherente de la red y propagar cambios; en el protocolo propuesto este proceso ocurre de forma completamente distribuida.

Continuidad Lógica La continuidad lógica es la capacidad del protocolo para mantener el flujo de información incluso ante cambios topológicos, como la desconexión de un nodo o la

degradación de un enlace (Karl & Willig, 2005). Constituye uno de los objetivos centrales del protocolo y una de las métricas de evaluación principal en las pruebas experimentales, midiendo tiempo de reconexión y paquetes perdidos durante eventos de reconfiguración.

Sockets y Comunicación TCP/UDP en Android Un socket permite a una aplicación enviar y recibir datos en red; en Android se implementa sobre la API estándar de Java (`java.net.Socket` para TCP y `java.net.DatagramSocket` para UDP) sin acceder a niveles inferiores de la pila (Android Developers, 2024). El protocolo usa sockets TCP para intercambio de datos entre nodos con ruta establecida, y sockets UDP para mensajes de descubrimiento y señalización topológica, aprovechando su bajo overhead y compatibilidad con broadcast en la subred local (Tanenbaum & Wetherall, 2011).

En definitiva, los conceptos y tecnologías revisados a lo largo de este apartado forman la base sobre la cual se sustenta el diseño del protocolo propuesto. Entender cómo funcionan las redes ad hoc y multisalto, qué estrategias existen para el enrutamiento y la prevención de ciclos, y qué herramientas ofrece la plataforma Android para la comunicación en red, permite tomar decisiones de diseño sólidas y evaluar con criterio el comportamiento del sistema bajo distintos escenarios de prueba.

Estado del arte

Ad hoc On-Demand Distance Vector (AODV) AODV calcula rutas bajo demanda difundiendo mensajes RREQ e instalando caminos de retorno RREP, usando secuencias para evitar ciclos (Perkins et al., 2003). Al exigir operación en la capa de red del SO, es inviable en Android no rooteado. El proyecto hereda su mitigación de fallos bajo demanda, trasladándola por completo a la capa de aplicación.

Optimized Link State Routing (OLSR) Protocolo proactivo que mapea rutas propagando continuamente el estado de los enlaces (Clausen & Jacquet, 2003). Al igual que AODV, su dependencia estricta de la capa de red impide su ejecución en Android retail. De OLSR se rescata únicamente el concepto de mantener y actualizar topologías locales eficientemente.

Meshtastic implementa redes multisalto de largo alcance usando tecnología LoRa para emergencias (Meshtastic Project, 2023). Su principal limitación es la dependencia de hardware especializado y una tasa de transferencia reducida (del orden de Kbps), lo que lo hace poco práctico para comunicación en tiempo real frente al Wi-Fi estándar.

Bridgefy y Briar Bridgefy es una plataforma de mensajería descentralizada que emplea Bluetooth y Wi-Fi para comunicación sin internet, pero investigadores de la ETH Zúrich identificaron vulnerabilidades serias en su protocolo, incluyendo suplantación de identidad y ausencia de confidencialidad (Albrecht et al., 2021); adicionalmente, su código cerrado dificulta su uso académico. Briar, por su parte, es una aplicación de código abierto con cifrado extremo a extremo sobre Bluetooth y Wi-Fi (Briar Project, 2021), pero su enrutamiento multisalto está restringido a contactos previamente conocidos, limitando la cobertura en redes con nodos desconocidos. Ambas confirman la viabilidad técnica de la comunicación descentralizada sobre hardware móvil estándar, pero ninguna aborda la reconfiguración topológica distribuida ante cambios dinámicos.

BitChat es un protocolo de mensajería descentralizado y cifrado de extremo a extremo que utiliza una arquitectura híbrida de red en malla sobre Bluetooth Low Energy (BLE) y el protocolo Nostr para el enrutamiento global (Biagioni, 2025). A diferencia de las soluciones centralizadas, opera sin identificadores persistentes ni servidores de coordinación, permitiendo la comunicación en entornos sin acceso a internet o redes celulares. Aunque implementa un sistema multihop de hasta 7 saltos, su dependencia del estándar Bluetooth limita el ancho de banda efectivo y la profundidad de la red, lo que lo posiciona como una solución de baja tasa de bits frente a las capacidades teóricas del Wi-Fi.

Análisis comparativo Protocolo / App Hardware AODV / OLSR Wi-Fi Estándar Si (Capa Red) Radios LoRa No Bridgefy / Briar Bluetooth/Wi-Fi No (Capa App) Bluetooth Mesh No (Capa App) Propuesta actual Wi-Fi Estándar No (Capa App)

Marco legal

El desarrollo del proyecto involucra software, datos de usuario y comunicaciones inalám-

bricas, por lo que debe alinearse con el marco normativo colombiano vigente en materia de protección de datos personales, TIC y derechos de autor.

Ley 1581 de 2012 — Protección de datos personales Regula el tratamiento de datos personales en Colombia, exigiendo estándares de confidencialidad (Congreso de la República de Colombia, 2012). Para el proyecto, obliga a que los mensajes ajenos no sean retenidos prolongadamente por los nodos intermediarios, descartándolos de la memoria base del dispositivo tan pronto concluya el salto estructural.

Ley 1341 de 2009 — Ley de TIC Esta ley establece los principios del sector TIC en Colombia y promueve el acceso universal a las comunicaciones, la investigación tecnológica y la neutralidad tecnológica (Congreso de la República de Colombia, 2009). El proyecto se alinea con esta ley en tanto el protocolo propuesto busca extender la conectividad en escenarios donde los servicios convencionales no están disponibles.

Diseño metodológico

Para el desarrollo del presente proyecto se implementará una metodología incremental. Este enfoque de ingeniería de software se basa en la construcción del sistema a través de entregas parciales y sucesivas, denominadas incrementos. En cada incremento se realiza un ciclo que abarca el análisis, diseño, implementación y pruebas de un conjunto específico de requisitos funcionales, lo que permite validar progresivamente el protocolo propuesto. A diferencia de un modelo lineal tradicional, el desarrollo iterativo facilita la retroalimentación continua y la detección temprana de errores conceptuales o técnicos. Esta característica resulta fundamental para el diseño de un protocolo sobre capas de abstracción no nativas (capa de aplicación en Android), donde limitaciones imprevistas de las APIs o comportamientos anómalos del entorno de red requieren ajustes dinámicos de diseño sin afectar todo el ciclo de vida del proyecto.

El equipo de trabajo está conformado por integrantes que asumirán roles complementarios durante las iteraciones, siguiendo lineamientos inspirados en marcos de trabajo ágiles adaptados al desarrollo académico: Equipo de Desarrollo (Estudiantes proyectantes): • Juan Carlos Clavijo: Responsable del diseño lógico del protocolo, la arquitectura concurrente, el desarrollo de los algoritmos de enrutamiento (tablas de salto) y la lógica de detección de fallos mediante heartbeats. • Brandon Stiven Ganzo: Encargado de la implementación de la capa de comunicación de bajo nivel (UDP broadcast), el túnel de control TCP, el desarrollo del subsistema de mensajería y el diseño de la interfaz gráfica de usuario. Director del Proyecto: Cumple el rol de orientador metodológico y técnico. Su responsabilidad es velar por el rigor del proceso, validar la coherencia conceptual del diseño con la teoría de redes distribuidas y evaluar los incrementos desarrollados, asegurando que cada entrega aporte al cumplimiento de los objetivos específicos.

Fases del proyecto y desarrollo iterativo El proyecto se organizará en cuatro iteraciones o incrementos principales correspondientes al desarrollo del protocolo y su aplicación en el sistema demostrativo. Cada fase culmina con un entregable verificable.

Incremento 1: Análisis y diseño de la arquitectura base Este incremento establece los cimientos lógicos del sistema. Incluye la revisión de los estándares técnicos (Wi-Fi Manager, Wi-Fi

Direct) y protocolos de red ad hoc de referencia (AODV, OLSR), definiendo cómo se aplicarán las reglas de enrutamiento distribuidas en un entorno sin infraestructura. Actividades: Identificación de requerimientos, análisis de restricciones en APIs de Android (desde Android 12 en adelante), diseño lógico de las estructuras de datos de enrutamiento y concepción arquitectónica del sistema operativo concurrente. Entregables: Documento de requerimientos priorizado, diagramas de clases y de secuencia del protocolo de comunicación base.

Incremento 2: Descubrimiento de nodos y topología inicial Se procede con la implementación de los servicios de anuncio y la formación de una red local autónoma mediante la asignación de roles de acceso (SoftAP) y puntos cliente. Actividades: Desarrollo del módulo de anuncio por broadcast UDP, programación de los sockets TCP para el establecimiento de enlaces seguros y creación de la interfaz (UI) básica de inicio de sistema en Android. Entregables: Aplicación capaz de identificar vecinos en el mismo espectro de cobertura Wi-Fi y establecer un enlace de una vía entre dos nodos sin intervención externa.

Incremento 3: Enrutamiento y mensajería en red multisalto Este incremento corresponde al núcleo del proyecto. Sobre los nodos enlazados, se diseñan e implementan las tablas de estados de los vecinos (routing tables) que posibilitan extender la comunicación a nivel escalonado. Se integran mensajes de control inter-nodos (señalizadores) y mensajes de carga útil (texto o chat de usuario). Actividades: Desarrollo del algoritmo de propagación de rutas basándose en TTL y secuencias numéricas, control de ciclos infinitos, e implementación del intercambio estructurado de mensajes entre nodos no directos. Entregables: Software beta con capacidad de intercambio de mensajes bidireccional abarcando una cadena mínima de 3 dispositivos intermedios.

Incremento 4: Reconfiguración topológica, validación y escalamiento Fase final de análisis dinámico donde ocurren las interrupciones para validar las facultades resilientes. Se incorporan las funciones lógicas de detección de desconexión abrupta por medio de "heartbeats regulares o fallos de envíos. Actividades: Programación del procedimiento de purga de rutas rotas tras intervalos temporizados, programación de re-generación e incrustación de métricas automáticas, y pruebas empíricas experimentales completas. Entregables: Versión estable con reconexión reactiva

demostrable ante apagados sistemáticos, recolección depurada de datos empíricos e informe con los manuales/documentación. Para propósitos de organización, la Tabla 3.1 detalla las acciones condensadas de estos procesos en sus respectivos incrementos funcionales.

Incremento Tareas principales Levantamiento de requerimientos, análisis de restricciones en Android, diseño de arquitectura concurrente.

Programación UDP broadcast, establecimiento de Conectividad túnel de transmisión TCP-IP en dispositivos en Base corto rango. 3. Subsistema Programación lógica del intercambio de tablas de (routing), sistema de contención anti-ciclos e enrutamiento interfaz de mensajería (chat). 4. Estabilidad Lógica de latidos, sistema de reconstrucción y Pruebas automática tras la caída de puentes e inyección de datos experimentales. 1. Análisis y Diseño

Requisitos documentados, diagramas UML / de Secuencia. Módulo de escaneo emparejado entre 2 nodos. Red conformable por ≥ 3 saltos simultáneos y funcionales. Versión final, registros empíricos, manual y documentaciones finales.

Técnicas e instrumentos de validación experimental Dado el enfoque puramente tecnológico experimental, la validación del trabajo reposa sobre el desempeño real del producto derivado de la programación en laboratorio y no de análisis sociodemográficos u observación participativa teórica. Los ensayos se desarrollarán creando redes preconfiguradas y redes emergentes (espontáneas) empleando múltiples terminales móviles controlados. Se evaluará el cumplimiento de cada requisito funcional establecido en el incremento preliminar. Las variables observadas consistirán en: Tiempo de reconexión adaptativo: Fracción en milisegundos que la red invertirá hasta encontrar y propiciar un nuevo recorrido efectivo cuando el enlace principal se interrumpe artificialmente. Estabilidad operativa y profundidad: Verificación del máximo de escalonamiento estructural (saltos) permitidos en función del ancho de banda y latencia originada por la retransmisión computarizada de la pasarela de Android.

Desarrollo del sistema

El desarrollo del protocolo PCT se ejecutó siguiendo la metodología incremental del ante-

proyecto, organizada en cuatro sprints académicos de tres semanas cada uno (12 semanas totales). Cada sprint corresponde a un incremento funcional verificable: diseño formal, descubrimiento y enlace L2, enrutamiento multisalto L3 y reconfiguración topológica con validación empírica. La implementación se consolidó en la biblioteca Maven `co.uan.pct:core` (módulo `lib-pct-core`, versión 2.x) y en la aplicación demostrativa `pctmesh` (`app-demo`), desplegable como APK de depuración en dispositivos Android 12+ (API 31) sin acceso *root*.

La arquitectura de software separa responsabilidades en capas alineadas con la especificación wire (`docs/protocol/`): capa L1 (Wi-Fi Direct GO, STA legacy, DNS-SD), capa L2 (TCP control :8765 y datos :8766, NeighborRegistry), capa L3 (RoutingTable, RouteOrchestrator, ForwardWorker) y capa de aplicación (PctNode, eventos de usuario, UI de depuración). El punto de entrada público es la interfaz PctNode, instanciada internamente como PctNodeImpl, la cual orquesta el ciclo `init` $\rightarrow$ `start` $\rightarrow$ bootstrap automático $\rightarrow$ operación $\rightarrow$ `close`.

Entorno de desarrollo y dispositivos de laboratorio

El entorno de compilación utiliza Gradle 8.x, Kotlin 1.9+, Android Gradle Plugin y JDK 17. La biblioteca core se publica como artefacto AAR/JAR consumible desde el módulo de aplicación. Las pruebas unitarias se ejecutan con `./gradlew :lib-pct-core:test` en JVM; las pruebas de integración y sistema requieren dispositivos físicos conectados vía ADB.

Los terminales de laboratorio clasificados como dispositivos clase α son: Samsung Galaxy A24 (R58W6053SSW), designado habitualmente como nodo ROOT en escenarios de dos y tres saltos, y Vivo Y21s (3435708850002SR), empleado como nodo hijo, BRIDGE o LEAF según el experimento. Ambos cumplen el CDD de Android 12: RAM $\geq$ 2 GB, Wi-Fi Direct mandatorio y resolución mínima 426 dp $\times$ 320 dp. La captura de evidencia empírica se realiza con `adb logcat -s PctMesh`, filtrando la etiqueta definida en PctNodeImpl.

Sprint 1 — Análisis y diseño de la arquitectura base (semanas 1–3)

Objetivos del sprint

Establecer los cimientos conceptuales y formales del protocolo antes de escribir código de producción: levantamiento de requisitos funcionales y no funcionales, análisis de restricciones de plataforma Android, diseño de la arquitectura concurrente y redacción de la especificación wire inicial. Este sprint materializa el Incremento 1 del anteproyecto.

Actividades realizadas

Durante las primeras tres semanas se revisaron los estándares IEEE 802.11, las APIs `WifiP2pManager`, `ConnectivityManager.requestNetwork` y el mecanismo DNS-SD sobre Wi-Fi Direct. Se estudiaron protocolos de referencia (AODV, OLSR) para extraer patrones aplicables en capa de aplicación —descubrimiento bajo demanda, tablas de rutas locales, prevención de ciclos— descartando su implementación nativa por inviabilidad en Android retail.

Se redactó el documento de requisitos (`docs/protocol/01-requirements.md`) con identificadores RF-01 a RF-15, el modelo de enlace dual GO+STA (`04-link-model-go-legacy.md`), la arquitectura de componentes (`03-architecture.md`) y las máquinas de estado de bootstrap (`08-state-machines.md`). Se definió el formato binario de tramas TCP con prefijo PCT1 (`06-tcp-messages.md`) encabezado fijo de 12 bytes y catálogo de tipos HELLO, JOIN_COMMIT, TOPO_UPDATE, PING/PONG, NODE_DOWN y DATA.

Artefactos entregados

Artefacto	Descripción
Spec v0.1	19 documentos en <code>docs/protocol/</code>
Matriz EXP	Hipótesis H1–H5 y experimentos EXP-01..06 (<code>11-validation-matrix.md</code>)
Prototipo EXP-01	Módulo <code>proto-exp01-gostat</code> : dos GO + STA, sin L2 formal
Diagramas	Secuencia bootstrap <code>ROOT←hijo</code> ; capas L1/L2/L3

Decisiones de diseño clave

Se adoptó el principio de *continuidad lógica*: todo nodo mantiene GO propio (SoftAP P2P)

incluso cuando actúa como STA del padre, de modo que pueda aceptar hijos downstream sin reiniciar el SSID del árbol. Se definió el rol BRIDGE para nodos con padre upstream y al menos un hijo downstream. La identidad de red es un UUID de 128 bits (`node_id`) propagado autoritativamente en HELLO wire, no en registros TXT truncados.

Resultados parciales del sprint

Al cierre de la semana 3 se contaba con especificación revisable por el director de proyecto, trazabilidad requisito→experimento y un prototipo monolítico EXP-01 capaz de demostrar H1 (GO+STA simultáneo) de forma manual. No existía aún NeighborRegistry ni canal TCP formal; el handshake se probaba con sockets ad hoc. El gate EXP-01 se declaró criterio bloqueante para iniciar el Incremento 2.

Desglose semanal del Sprint 1

Semana 1. Revisión bibliográfica aplicada (AODV, OLSR, Meshtastic, Bridgefy, Briar, Bit-Chat) y extracción de requisitos RF desde el anteproyecto. Identificación de restricciones Android 12: WifiP2pManager exige permisos de ubicación para descubrimiento; NEARBY_WIFI_DEVICES en API 33+; coexistencia GO+STA no garantizada en todos los fabricantes.

Semana 2. Modelado UML de clases preliminar: entidades PctNode, GoRepository, LegacyStaRepository, DnsSdRepository, TopologySnapshot. Definición del perfil mínimo de enlace (documento 04): prohibición de connect() P2P para topología; solo STA legacy downstream. Formato SSID DIRECT-PCT- + prefijo hex de node_id.

Semana 3. Redacción de catálogo de mensajes TCP (12 tipos wire). Validación teórica de tamaños de frame: HELLO 59 B, JOIN_COMMIT 40 B, JOIN_ACK 17 B, NODE_DOWN 52 B. Revisión con director de proyecto. Entrega de paquete docs/protocol/ v0.1.0-pre y cronograma de sprints 2–4.

Sprint 2 — Descubrimiento, bootstrap y capa de enlace L2 (semanas 4–6)

Objetivos del sprint

Implementar el Incremento 2: descubrimiento automático de nodos vecinos, formación de topología inicial padre-hijo y enlace TCP bidireccional confiable en dos canales. Entregable mínimo: dos dispositivos enlazados con HELLO/JOIN completado y UUID visible en UI, sin intervención manual del operador.

Módulos implementados

Se creó el módulo `lib-pct-core` con paquetes `internal.p2p` (`GoRepository`, `DnsSdRepository`, `P2pChannelHolder`), `internal.sta` (`LegacyStaRepository`) e `internal.tcp` (`PctFrameCodec`, `PctControlSocket`, `PctDataSocket`, `PctLinkSocket`). El bootstrap automático en `PctNodeImpl.bootstrap()` ejecuta la secuencia:

1. Escaneo DNS-SD de servicios `_pct-ctrl._tcp.local` (GO apagado temporalmente si hay candidatos sin registro activo).
2. Selección de padre con `ParentSelector` (menor hop, mayor `tree_version`, desempate lexicográfico por UUID).
3. Asociación STA al SSID/PSK legacy del GO padre vía `requestNetwork`.
4. `createGroup()` local para restaurar GO propio del nodo hijo.
5. Re-asociación STA al padre (nodo BRIDGE).
6. Anuncio `_pct-ctrl` con registro TXT (`PctInstanceCodec`).
7. Activación L2: `LinkOrchestrator.onStaConnected` hacia upstream y `onGoReady` para accept downstream.

Capa de enlace: dos canales TCP

El `LinkOrchestrator` gestiona workers concurrentes sobre `Dispatchers.IO`: `controlAcceptLoop` y `dataAcceptLoop` en el GO, `connectUpstream` tras STA, `controlKeepaliveLoop` con PING/PONG

periódico. Cada vecino queda registrado en NeighborRegistry con sockets de control y datos, estado de canal (DataChannelState) y UUID autoritativo del HELLO recibido.

El requisito de Network.bindSocket para el socket upstream fue incorporado tras detectar self-connect a 192.168.49.1 cuando Android enrutaba por la interfaz P2P en lugar de la red STA activa. PctLinkSocket encapsula connect() y bindSocket() sobre el Network obtenido de LegacyStaRepository.

Problemas encontrados y correcciones

Durante el sprint se registraron fallos recurrentes documentados en la matriz de validación: (a) ambos nodos declarándose ROOT por escaneo prematuro sin _pct-ctrl activo — mitigado con espera de 10 s si hay peers P2P visibles pero cero candidatos DNS-SD; (b) UUID truncado a 32 caracteres hex en TXT — resuelto propagando UUID completo en payload HELLO de 59 bytes; (c) EOF en handshake por cierre prematuro del ServerSocket — resuelto manteniendo accept loops vivos mientras running=true; (d) conflicto GO encendido durante discoverServices — el escaneo DNS-SD se ejecuta con GO removido y se restaura tras seleccionar padre.

Resultados parciales del sprint

Criterio	Estado	Evidencia
EXP-02 DNS-SD en GO	Parcial	TXT parseado en emulador; pendiente repetición A24
EXP-01 dos GO enlazados	En curso	STA+HELLO observados; T_{join} por consolar
Tests PctFrameCodec	PASS	Round-trip HELLO 59 B, JOIN_COMMIT 40 B
UI debug bootstrap	PASS	Fase, acción, candidatos, SSID GO/STA

Al final de la semana 6 la aplicación demo mostraba en pantalla de depuración: rol inferido (ISLAND/ROOT/BRIDGE/LEAF), fase de bootstrap, conteo de candidatos pct=, estado GO/STA, enlaces ctrl/data abiertos y UUID copiable. El chat de usuario aún no operaba multisalto; correspondía al Sprint 3.

Desglose semanal del Sprint 2

Semana 4. Implementación de GoRepository (createGroup, removeGroup, observables goState) y LegacyStaRepository (requestNetwork, disconnect, estados Idle/Connecting/Connected/Error) Integración con P2pChannelHolder para registro de BroadcastReceiver y limpieza P2P en init/close.

Semana 5. DnsSdRepository: registro _pct-ctrl._tcp, descubrimiento, parseo TXT con PctTxtParser y PctInstanceCodec. Implementación de PctFrameCodec y tests round-trip. Primer bootstrap automático sin L2: ROOT si no hay candidatos; STA si hay padre seleccionado.

Semana 6. LinkOrchestrator + NeighborRegistry. Accept loops en puertos 8765/8766. connectUpstream con HELLO/JOIN_COMMIT. Corrección bindSocket. Pantalla MeshScreen con MeshViewModel enlazado a StateFlow de debug y topología. Primeras sesiones ADB A24+Vivo.

Secuencia de bootstrap en PctNodeImpl

La función bootstrap() ejecuta, en orden: (1) apagar GO residual si interfiere con escaneo; (2) discoverServices durante ventana configurable; (3) si hay candidatos _pct-ctrl, seleccionar padre y transitar a JOIN; si no, tras timeout convertirse en ROOT; (4) en JOIN: removeGroup, STA al padre, createGroup, re-STA, anuncio DNS-SD, link.onStaConnected y link.onGoReady; (5) publicar topología con rol BRIDGE o LEAF según presencia de hijos. Cada transición actualiza NodePhase (ISLAND, SCANNING, JOINING, ROOT, BRIDGE, LEAF) visible en UI.

Formato wire implementado (extracto)

Todos los frames usan magia ASCII PCT1, versión 1 y payload_len big-endian. El HELLO transporta sender_nid (16 bytes UUID binario), rol (ISLAND=0, LEAF=1, BRIDGE=2, ROOT=3), parent_nid, epoch, tree_version, hop y capabilities. Esta estructura fija de 59 bytes totales elimina ambigüedad de parseo respecto a implementaciones basadas en JSON o texto libre.

Sprint 3 — Enrutamiento multisalto y mensajería de usuario (semanas 7–9)

Objetivos del sprint

Materializar el Incremento 3: tablas de rutas locales, propagación TOPO_UPDATE, reenvío de mensajes de aplicación a través de nodos no adyacentes y validación de cadena mínima de tres dispositivos. El núcleo algorítmico reside en RouteOrchestrator y RoutingTable.

Implementación de la capa de red (L3)

RoutingTable almacena entradas RoutingEntry con destino UUID, siguiente salto (next_hop), métrica de saltos, estado (ACTIVE/STALE) y tree_version. Las rutas directas a vecinos se derivan de NeighborRegistry con hop=1. RouteSyncWorker procesa tramas TOPO_UPDATE recibidas en canal de control, fusionando anuncios con reglas de precedencia por versión de árbol y TTL decreciente para evitar inundación.

ForwardWorker opera exclusivamente sobre el canal de datos (:8766). Ante un PctUserEnvelope cuyo destino no es local, consulta la tabla, decrementa hop_limit, añade el UUID propio a path_trace y reenvía al next_hop. Si hop_limit llega a cero o el UUID ya figura en path_trace, el paquete se descarta (anti-ciclo).

La API pública expone sendUser(destinationNid, payload) y emite PctEvent.UserMessage en destino final. La UI MeshScreen integra lista de vecinos, tabla de rutas y campo de envío de prueba.

Pruebas internas del sprint

Las pruebas unitarias PctNidTest verifican normalización de UUID (con y sin guiones). PctNetworkHelperTest valida extracción de gateway IPv4 desde NetworkCapabilities de la red STA. PctFrameCodecTest cubre codificación/decodificación de todos los tipos de mensaje implementados.

Las pruebas de integración L3 en banco (dos JVM no aplican; requieren hardware) se planificaron como EXP-03: topología A (ROOT) ← B (BRIDGE) ← C (LEAF), envío DATA de C hacia A con reenvío en B.

Resultados parciales del sprint

Entregable	Estado	Observación
Tablas locales AC-TIVE	Implementado	Visibles en RouteSnapshot UI
TOPO_UPDATE gossip	Implementado	TTL configurable en PctConfig
ForwardWorker DA-TA	Implementado	Separado de canal control
EXP-03 cadena 3 nodos	Pendiente	Requiere tercer dispositivo o emulador
EXP-04 unión tardía	Diseñado	Procedimiento en matriz §2.4
EXP-06 _pct-seek	Parcial	Servicio definido; JOIN_OFFER en curso

Lecciones del sprint

La separación estricta de canales control/datos permitió saturar el canal de datos con mensajes de chat de prueba sin bloquear PING de keepalive — requisito explícito de la Fase A del roadmap de implementación. La topología publicada en TopologySnapshot se recalcula reactivamente combinando flujos neighbors y routes, evitando inconsistencias entre UI y estado interno.

Desglose semanal del Sprint 3

Semana 7. Diseño de RoutingTable y RoutingEntry. Rutas directas desde vecinos L2. Codificación TOPO_UPDATE con TTL y lista de prefijos anunciados. Integración en RouteOrchestrator.handl

Semana 8. ForwardWorker: decisión entregar local vs reenviar según destinationNid. Envelope PctUserEnvelope con hop_limit, path_trace y payload opaco. Emisión PctEvent.UserMessage en entrega final.

Semana 9. UI chat de prueba; observable routes en pantalla; pruebas de regresión JUnit ampliadas. Simulación manual de cadena A-B-C rotando roles entre dos dispositivos (modo degradado sin tercer terminal). Documentación 15-network-layer.md y 16-packet-envelope.md alineadas al código.

Modelo de datos de topología publicada

TopologySnapshot contiene nodo self (nodeId, role, hop) y lista de children conocidos downstream. Se combina con NeighborSnapshot (vecinos L2 con UUID, IP, estado canal datos) y RouteSnapshot (destino, next hop, hop count, estado ACTIVE/STALE). La UI de depuración muestra simultáneamente las tres vistas para correlacionar fallos de enrutamiento con fallos de enlace.

Sprint 4 — Reconfiguración topológica, endurecimiento y validación (semanas 10–12)

Objetivos del sprint

Completar el Incremento 4: detección de fallos por heartbeats, propagación NODE_DOWN, reconvergencia tras caída de nodo BRIDGE, recolección de métricas empíricas y estabilización de la versión demo para defensa de grado.

Mecanismos de resiliencia

El controlKeepaliveLoop emite PING periódico hacia vecinos upstream y downstream. Tras config.pingTimeoutMs sin PONG, el vecino se marca degradado y se programa reconexión upstream (upstreamReconnectJob) sin destruir GO local. NODE_DOWN (52 bytes payload) informa a vecinos de la caída de un nexo; las rutas que dependían de ese salto transitan a STALE hasta nueva convergencia.

El concepto de session_epoch (campo en HELLO y JOIN) permite que conversaciones activas sobrevivan reconfiguraciones menores: si el epoch no cambia tras re-JOIN, la capa de aplicación no reinicia estado de chat. EXP-05 valida H5.

DataChannelRecoveryWorker (negociación DATA_CHANNEL_OPEN/ACK por canal control) reabre :8766 caído sin reiniciar bootstrap L1.

Endurecimiento y deuda técnica

Persistencia de node_id entre reinicios queda como deuda (UUID regenerado en cada init). Cifrado E2E fuera de alcance (no entregable). El servicio _pct-seek y JOIN_OFFER/JOIN_ACCEPT

completos están especificados en `09-join-protocol.md` con implementación parcial para escenarios ISLAND.

Resultados parciales del sprint

Experimento	Estado	Notas
EXP-05	caída	Planificado
BRIDGE		Requiere 3 dispositivos + apagado simulado
EXP-01 gate	En curso	Bloqueante para cierre experimental
Reconexión ups-tream	Implementado	bindSocket + backoff
Log empírico	Plantilla	Formato markdown en matriz §4

Desglose semanal del Sprint 4

Semana 10. `controlKeepaliveLoop`: intervalo `pingIntervalMs`, timeout `pingTimeoutMs`. Marcado de vecino degradado. `upstreamReconnectJob` con backoff exponencial acotado.

Semana 11. `NODE_DOWN`: generación ante cierre de socket upstream; propagación limitada por TTL de control. `DATA_CHANNEL_OPEN/ACK/RESET` para recuperación de :8766. Deprecación de UDP HELLO provisional (HelloHub).

Semana 12. Consolidación APK debug; scripts ADB documentados en anexo; preparación plantillas EXP-01..06 para sesión de mediciones. Revisión de deuda técnica (persistencia UUID, `JOIN_OFFER` completo, cifrado).

Aplicación demostrativa `pctmesh`

El módulo `app-demo` consume `lib-pct-core` vía Gradle. `MainActivity` solicita permisos (`PctPermissions`) y lanza `MeshScreen` con Jetpack Compose. `MeshViewModel` observa `PctNode.topology`, `neighbors`, `routes`, `debug` y `events`. Acciones expuestas: `start()`, `close()`, envío de mensaje de prueba a UUID destino. La pantalla de depuración muestra candidatos DNS-SD parseados, contadores `peerCount/pctCtrlSeen` y estado de enlaces `ctrl/data` — instrumentación esencial para correlacionar logs con comportamiento visible.

Evolución del repositorio y trazabilidad

La migración desde proto-exp01-gostat hacia lib-pct-core preservó el comportamiento L1 validado en EXP-01 y añadió capas formales. La tabla siguiente resume la trazabilidad sprint→módulo→experimento.

Sprint	Módulos principales	Versión lib	EXP asociado
1	docs/protocol/*	—	Diseño
2	p2p/, sta/, tcp/, link/	1.0–1.5	EXP-01, EXP-02
3	route/, PctNodeImpl	1.8–2.0	EXP-03, EXP-04, EXP-06
4	keepalive, recovery, métricas	2.0+	EXP-05

Síntesis del desarrollo

Al cierre del periodo de desarrollo documentado, el sistema implementa bootstrap automático completo, enlace L2 dual TCP, tablas de rutas L3, reenvío de mensajes de usuario y mecanismos de keepalive/recuperación parcial. Los sprints 1–2 aportaron la base formal y el enlace vecino; el sprint 3 habilitó la extensión multisalto lógica; el sprint 4 concentró la resiliencia y la instrumentación para mediciones. La validación empírica consolidada se presenta en el capítulo siguiente.

Resultados y análisis de resultados

Marco de evaluación experimental

La evaluación sigue la matriz de validación (docs/protocol/11-validation-matrix.md), que define cinco hipótesis (H1–H5) y seis experimentos (EXP-01..06) con criterios de aceptación binarios (PASS/FAIL) y métricas cuantitativas. Las variables dependientes principales son: tiempo de unión (T_{join}), tiempo de unión en cadena ($T_{\text{join_chain}}$), latencia extremo a extremo ($T_{\text{data_e2e}}$), tiempo de reconfiguración (T_{reconfig}) y profundidad topológica máxima alcanzada.

Los instrumentos de medición son: (1) logs estructurados PctMesh con marcas temporales implícitas de logcat; (2) snapshots de UI (PctDebugSnapshot) capturables en pantalla; (3) tests JUnit automatizados; (4) tabla de registro empírico por experimento (plantilla §4 de la matriz).

Resultados de pruebas unitarias y de componente

Las pruebas aisladas ejecutadas en CI/local sin hardware validan la correctitud del protocolo wire y utilidades de red. No sustituyen EXP-01..06 pero garantizan regresión ante cambios en códec o helpers.

Suite	Resultado	Cobertura funcional
PctFrameCodecTest	PASS	HELLO 59 B, JOIN 40/17 B, TOPO, PING/PONG, DATA
PctNidTest	PASS	Normalización UUID 32/36 chars
PctNetworkHelperTest	PASS	Gateway STA IPv4
PctCoreTest	PASS	Smoke API pública

El round-trip de HELLO verifica magia PCT1, versión 1, msg_type=0x01 y payload de 47 bytes con sender_nid, rol enum, parent_nid, epoch y hop. Errores de longitud o magia inválida lanzan excepción en decode, impidiendo estados corruptos en L2.

EXP-01 — Interconexión de dos Group Owners

Configuración y procedimiento

Dispositivo D1 (Samsung A24): nodo ROOT. Dispositivo D2 (Vivo Y21s): nodo hijo. Secuencia: D1 `start()` → espera anuncio `_pct-ctrl` → D2 `start()`. D2 debe descubrir D1, asociar STA, recrear GO, re-asociar STA, completar TCP HELLO upstream y JOIN_COMMIT.

Criterios de aceptación

1. D2 asocia STA al SSID legacy de D1 (estado `StaState.Connected`).
2. D2 mantiene GO propio activo tras re-asociación (H1).
3. HELLO L2 completado; UUID de D1 visible en UI de D2.
4. $T_{\text{join}} < 15\,000$ ms (umbral teórico del anteproyecto).

Resultados observados

Indicador	Criterio	Observado
STA asociada al padre	Sí	Sí (intermitente)
GO hijo post-STA	Sí	Sí
HELLO L2 + UUID completo	Sí	Sí tras fix wire
JOIN_COMMIT / ACK	Sí	Parcial
T_{join} (ms)	$< 15\,000$	Por medir

Análisis EXP-01

Los registros de laboratorio muestran que la fase más variable es el escaneo DNS-SD inicial: si D2 escanea antes de que D1 anuncie `_pct-ctrl`, ambos convergen a ROOT (H2 falla). La espera de 10 s con peers visibles mitiga pero no elimina la condición de carrera. Tras sincronizar manualmente el orden de `start()`, STA y HELLO se observaron de forma repetible. El self-connect a `192.168.49.1` se diagnosticó cuando `local=remote` en log; la corrección `bindSocket` en red STA resolvió el patrón en pruebas posteriores.

EXP-01 permanece como *gate* del proyecto: sin PASS estable, los experimentos multisalto no son concluyentes.

EXP-02 — DNS-SD en Group Owner sin Group Membership

Objetivo

Confirmar que `addLocalService` opera exclusivamente con GO activo y que D2 puede `discoverServices()` sin `connect()` P2P, recibiendo TXT completo con `nid`, `go_ssid` y `cp` (código de perfil).

Resultados

En pruebas controladas, D1 con `createGroup()` + registro `_pct-ctrl` permite a D2 parsear candidatos en `ParentSelector`. El campo TXT limitado a 198 bytes obliga a no depender de UUID completo en DNS; el UUID autoritativo se obtiene del HELLO TCP (59 bytes). Resultado: PASS en parseo de candidatos; FAIL intermitente en descubrimiento si GO de D2 interfiere — mitigación documentada en Sprint 2.

EXP-03 — Cadena de tres nodos

Topología

A (ROOT) $\leftarrow$ B (BRIDGE) $\leftarrow$ C (LEAF). C envía DATA de aplicación hacia A; B reenvía por `ForwardWorker` sin procesar payload.

Criterios y métricas teóricas

Entrega en A con `dst_nid` correcto; $T_{\text{join_chain}} \leq 30000$ ms; $T_{\text{data_e2e}} \leq 500$ ms en laboratorio ideal; 0 % pérdida en 100 mensajes de prueba.

Estado actual

Implementación L3 completa en código; validación física pendiente de tercer terminal o rotación de roles en sesiones seriales. Las tablas de rutas en B deben listar A como `next_hop` hacia destinos upstream y C como vecino downstream. `TOPO_UPDATE` debe converger antes del primer `sendUser`.

Análisis anticipado

La latencia en cadena de tres saltos estará dominada por scheduling Android en Dispatchers . IO y contención Wi-Fi en banda 2.4 GHz compartida. Se espera T_{data_e2e} superior al umbral ideal de 500 ms en hardware clase α , sin invalidar la hipótesis H3 (conectividad lógica multisalto).

EXP-04 — Unión tardía de nodo ISLAND

Procedimiento

Con $A \leftrightarrow B$ operativos ≥ 60 s, encender C en modo ISLAND con `_pct-seek`. B responde JOIN_OFFER; C completa STA y JOIN. Verificar que SSID de A y B no cambian (RF-14).

Resultados parciales

Procedimiento especificado; JOIN_OFFER wire parcialmente implementado. Análisis: la unión tardía es crítica para escenarios post-desastre donde nodos se encienden de forma asíncrona; el diseño GO persistente en cada nodo evita reconfiguración de SSID raíz.

EXP-05 — Caída de nodo puente

Procedimiento

Topología $A \leftarrow B \leftarrow C$. Apagar aplicación o radio de B. C transita a ISLAND; A recibe NODE_DOWN. C re-JOIN directo a A si alcanza radio. Medir $T_{reconfig}$ y estabilidad de `session_epoch`.

Métricas teóricas

$T_{reconfig} \leq 15\,000$ ms; epoch estable en conversación activa (H5).

Análisis

La caída de BRIDGE es el escenario de mayor valor para continuidad lógica post-desastre. El mecanismo de STALE routes + reconexión upstream implementado en Sprint 4 aborda el caso; falta evidencia numérica de defensa. Se recomienda repetir con carga de mensajes activa durante

apagado de B.

EXP-06 — ISLAND cerca de LEAF

Nodo C ISLAND entra en radio de LEAF L ($A \leftarrow B \leftarrow L$). L detecta `_pct-seek` de C y actúa como ofertante sin reiniciar GO. Valida diseño homogéneo GO en todos los roles. Estado: especificado; prueba física pendiente.

Análisis de restricciones de plataforma observadas en desarrollo

Durante los cuatro sprints se confirmaron restricciones documentadas en `02-platform-constraints.md`. (1) el framework Android no expone reenvío IP en user space — todo multisalto ocurre en aplicación; (2) Wi-Fi Direct y STA compiten por antena en 2.4 GHz, introduciendo latencia en `re-STA post-createGroup`; (3) Samsung y Vivo difieren en tiempos de `removeGroup/createGroup` (A24 típicamente 2–4 s, Y21s 3–6 s); (4) el límite de TXT DNS-SD (198 bytes) impide empaquetar credenciales y metadatos extensos — diseño híbrido TXT + HELLO TCP.

Estas restricciones no invalidan el protocolo pero explican variabilidad en T_{join} entre fabricantes y la necesidad de umbrales generosos (15 s) en criterios de aceptación.

Análisis integrado de logs de laboratorio

La etiqueta `PctMesh` concentra evidencia diagnóstica. Patrones identificados:

Patrón en log	Interpretación
pct=0 candidatos=0 con peers	Escaneo prematuro; aumentar delay o reordenar
P2P	start()
local=192.168.49.1	Self-connect; aplicar bindSocket STA
remote=192.168.49.1	
UUID con ceros en cola	TXT truncado; usar HELLO wire
EOF tras HELLO	Accept loop cerrado; revisar ciclo de vida L2
[L2] GO ready - accept	Bootstrap L2 downstream OK
:8765/:8766	
upstream connect + IP padre	Enlace STA correcto

Contraste con objetivos específicos del anteproyecto

OE	Objetivo específico	Grado cumplimiento
OE1	Diseño protocolo distribuido	Alto — spec + lib
OE2	Estructura comunicación lógica	Alto — L2/L3
OE3	Implementación Android demo	Alto — APK + UI
OE4	Evaluación laboratorio	Medio — EXP en curso

OE4 permanece en grado medio hasta completar mediciones de T_{join} , T_{reconfig} y cadena tres nodos con evidencia tabulada.

Discusión general

Los resultados confirman la viabilidad técnica de operar un protocolo multisalto en capa de aplicación sobre Wi-Fi Direct GO + STA legacy sin *root*, alineado con H1 y parcialmente con H2. La complejidad real reside en la coexistencia de interfaces (P2P + infraestructura) y en las condiciones de carrera del bootstrap DNS-SD, no en el modelo lógico de tablas y reenvío.

Frente al estado del arte (AODV, OLSR, Meshtastic, Bridgefy), PCT sacrifica integración en capa de red a cambio de despliegue en hardware retail. Frente a prototipos académicos UDP-only, el canal dual TCP aporta confiabilidad y separación control/datos verificable en logs.

Las limitaciones reconocidas — latencia del SO, UUID efímero por sesión, ausencia de cifrado E2E, validación parcial de EXP-03..06 por restricción de hardware — delimitan el alcance de las conclusiones finales, las cuales se consolidarán cuando las métricas empíricas estén completas.

Síntesis de resultados

El desarrollo por sprints produjo una biblioteca funcional con trazabilidad documentación→código→experimento. Las pruebas unitarias PASS garantizan integridad del wire format. EXP-01 y EXP-02 aportan evidencia cualitativa de enlace vecino; EXP-03..06 tienen implementación de soporte en L3/L4 y procedimientos definidos, pendientes de mediciones numéricas finales en sesión de defensa. El análisis de logs proporciona un marco reproducible para depuración y extensión futura del protocolo PCT.

Referencias bibliográficas

Referencias

- [1] Akyildiz, I. F., Wang, X., & Wang, W. (2005). Wireless mesh networks: A survey. *Computer Networks*, 47(4), 445–487. <https://doi.org/10.1016/j.comnet.2004.12.001>
- [2] Albrecht, M. R., Hernandez-Guzmán, J., & Poettering, B. (2021). SoK: Secure messaging. *Proceedings of the 2021 IEEE Symposium on Security and Privacy*, 1003–1020.
- [3] Android Developers. (2024). *Network overview*. <https://developer.android.com/develop/connectivity/network-access>
- [4] Appiahene, P., et al. (2019). Performance analysis of Wi-Fi networks in crowded environments. *Journal of Network and Computer Applications*.
- [5] Biagioni, E. S. (2025). Early review of the BitChat protocol. *ICNC*.
- [6] Briar Project. (2021). *Briar: Secure messaging, anywhere*. <https://briarproject.org>
- [7] Clausen, T., & Jacquet, P. (2003). *RFC 3626: OLSR*. IETF.
- [8] Congreso de Colombia. (2012). *Ley 1581 de 2012*. <https://www.funcionpublica.gov.co/>
- [9] Congreso de Colombia. (2009). *Ley 1341 de 2009*. <https://www.mintic.gov.co/>
- [10] Conti, M., & Giordano, S. (2014). Mobile ad hoc networking. *IEEE Communications Magazine*, 52(1), 85–96.
- [11] Google. (2021). *Android 12 CDD*. <https://source.android.com/docs/compatibility/12/android-12-cdd>
- [12] Gupta, P., & Kumar, P. R. (2000). The capacity of wireless networks. *IEEE Transactions on Information Theory*, 46(2), 388–404.

- [13] IEEE. (2020). *IEEE Std 802.11-2020*. <https://doi.org/10.1109/IEEESTD.2021.9363693>
- [14] Karl, H., & Willig, A. (2005). *Protocols and architectures for wireless sensor networks*. Wiley.
- [15] Maskrey, A. (1993). *Los desastres no son naturales*. LA RED.
- [16] Meshtastic Project. (2023). <https://meshtastic.org>
- [17] Perkins, C. E. (2001). *Ad hoc networking*. Addison-Wesley.
- [18] Perkins, C. E., Belding-Royer, E. M., & Das, S. (2003). *RFC 3561: AODV*. IETF.
- [19] Sterbenz, J. P. G., et al. (2010). Resilience and survivability in communication networks. *Computer Networks*, 54(8), 1245–1265.
- [20] Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed systems: Principles and paradigms* (2nd ed.). Prentice Hall.
- [21] Tanenbaum, A. S., & Wetherall, D. J. (2011). *Redes de computadoras* (5th ed.). Prentice Hall.

Anexos

Anexo A. Especificación wire TCP PCT

Extracto de docs/protocol/06-tcp-messages.md.

Anexo B. Manual de despliegue en laboratorio

Instalación del APK, permisos de ubicación y Wi-Fi, secuencia ROOT → hijo, captura de logcat con etiqueta PctMesh.

Anexo C. Cronograma y costos del anteproyecto

Ver AnteProyecto-Final.pdf, capítulos 4 y 5.